Register Verification

RISCFORGE 1TOPS SoC

Watchdog Timer • Register Access Test • Bus Timeout Monitors

REG-VERIF-v1.0 | September 2026

Document Overview

This document covers three pieces of work carried out on the RISCFORGE 1TOPS SoC: the addition of a watchdog timer, a firmware-driven test that accesses every peripheral register through the real bus path, and a firmware-driven test of the bus timeout monitors that were added so that a slave which stops answering can no longer hang the CPU.

All three run on the real RV32 core after a cold boot from the Infineon S25FS512S flash model. No part of this work uses UVM; the testbenches are plain SystemVerilog with directed stimulus and a pass/fail scoreboard, matching the other functional testbenches in the project.

Field	Value
SoC	RV32IM_Zicsr single-core MCU, 32 KB ISRAM + 32 KB DSRAM, 64 B BootROM
Peripherals	I2C, Timer 0, UART, GPIO, SPI Master, Timer 1 (6 APB slots)
Bus fabric	AXI-Lite crossbar (2 SLV → 3 MST) + AXI-to-APB bridge (6 slaves)
Simulator	Cadence Xcelium 24.09-s001
Register test	`rtl/soc_regtest.f` → `soc_regtest_tb`
Monitor test	`rtl/soc_montest.f` → `soc_montest_tb`
Waveforms	written on every run (`*_tb.shm`), no command-line flag

Deliverable	Checks Run	Result	Section
Watchdog timer (`aon_timer`)	1 194	PASS	Section 2
Register access test	469	FAIL	Section 3
Bus timeout monitor test	321	PASS	Section 4

Part 1 — Watchdog Timer

The SoC had no watchdog, so any firmware stall or bus hang was unrecoverable without an external power-on reset. The OpenTitan `aon_timer` block was selected and vendored in rather than written from scratch, because it is a silicon-proven, independently verified IP with a published verification environment.

Why a Watchdog Is Required

The bus timeout monitors of Section 4 solve one specific problem: a slave that stops answering. They convert that into a precise exception so the CPU keeps running. They are blind to everything else. A

watchdog is the only mechanism in the design that recovers a part which is powered and clocked but is no longer making progress, and it is the only one that works when the CPU itself is the thing that is stuck.

The distinction matters because the two are often confused. A bus timeout monitor is a *fabric* guard: it watches one handshake and needs the CPU alive to take the resulting exception. A watchdog is a *system* guard: it lives in the always-on clock domain, it does not depend on the main clock, the bus or the CPU, and it asserts a reset when firmware stops petting it, whatever the reason. Neither substitutes for the other.

Cases that require a watchdog in this SoC

#	Failure case	What happens today	Bus mon.	Watchdog
1	Firmware poll loop with no exit	A wait on a status bit that never sets: `SR.TIP` after an I2C arbitration loss, `LSR.DR` when the far end never transmits, `STATUS` busy after a mis-configured SPI command. Every bus access completes normally, so the fabric sees a perfectly healthy system while the program makes no progress for ever	No	Yes
2	Boot-time SPI stall	The bootloader's `S_READ_FIFO` state waits on `PREADY` through its own master port, which bypasses the APB monitor entirely. A flash that fails to respond leaves the part stuck before the core is even released	No	Yes
3	Peripheral wedged after an abort	A UART timeout leaves `apb_to_obi` mid-handshake (see the finding in Section 4.4). The CPU recovers via the exception, but that peripheral stays unusable until something resets it	Partly	Yes
4	Exception storm	A single corrupted pointer produced 2 311 access faults in the register test. The handler resumes each time, so the program never stops — it simply never finishes either. No individual access is illegal for long enough to be noticed	No	Yes
5	Runaway control flow	Stack overflow, a corrupted return address, or a jump into data. Execution continues at an arbitrary address and may loop indefinitely inside valid memory without ever raising a bus error	No	Yes
6	CPU frozen on a stalled access	If a monitor threshold is ever mis-sized, or a fault appears on a path no monitor covers, the core stops mid-instruction. Nothing driven by the CPU can recover it, including any software-based supervision	Partly	Yes
7	Clock or reset disturbance	A peripheral left in an illegal state by a glitch, or a clock that stops. The watchdog runs on the always-on clock, so it keeps counting even when the main clock domain is gated or dead	No	Yes
8	Deployed operation	In the field there is no JTAG probe and no operator to press reset. Autonomous recovery is the only recovery	No	Yes

How the block covers them

`aon_timer` provides a two-stage watchdog on the always-on clock. The *bark* threshold raises an interrupt, giving firmware a chance to notice and record why it was late; the *bite* threshold asserts a reset request and recovers the part unconditionally. A `WDOG_REGWEN` lock lets firmware seal the configuration after boot

so that runaway code cannot disable the watchdog it is supposed to be caught by, and a pause-in-sleep control stops the counter during intentional low-power states so that sleeping is not mistaken for hanging. The same block also contains an independent wake-up timer, which is what brings the part out of those sleep states.

Note

Of the cases above, only numbers 3 and 6 are partly covered by the bus timeout monitors, and in both of those the monitor recovers the CPU while leaving the peripheral or the system state unusable. Every remaining case has no recovery path in the current design at all. That is the argument for the watchdog: it is not redundancy with the monitors, it is the only cover for a class of failure the monitors cannot observe.

IP Source

Field	Value
Repository	`https://github.com/lowRISC/opentitan`
Branch	`earlgrey_1.0.0`
Commit	`7d0af70`
IP path	`hw/ip/aon_timer`
IP version	2.0.0
Life-cycle stage	`D2S` / `V2S` (design and verification both signed off)
Licence	Apache-2.0; modification notices added to every file we touched
Direct link	`https://github.com/lowRISC/opentitan/tree/earlgrey_1.0.0/hw/ip/aon_timer`

UVM verification environment

The IP ships with a complete UVM DV environment in the same repository, so the block already has constrained-random and coverage-driven verification behind it, carried out by lowRISC and independent of anything done here:

Artefact	Location in the repository
UVM environment	`hw/ip/aon_timer/dv/env/`
Sequence library	`hw/ip/aon_timer/dv/env/seq_lib/`
Test list	`hw/ip/aon_timer/dv/tests/`
Testplan	`hw/ip/aon_timer/data/aon_timer_testplan.hjson`
Regression config	`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson`

The testplan covers the wake-up timer, the bark and bite watchdog thresholds, the `WDOG_REGWEN` lock, pause-in-sleep behaviour, the life-cycle escalation input, the interrupt and reset outputs, and the standard CSR suite (hardware reset, read/write, bit-bash and aliasing). Because the block is at `V2S`, that plan is signed off as complete rather than in progress.

Physical design

`aon_timer` is part of the OpenTitan Earl Grey top level, which has been taken through a complete RTL-to-GDS implementation flow and taped out. The block is therefore physical-design verified: synthesis, place and route, timing closure and DRC/LVS sign-off have all been carried out on this RTL by the OpenTitan programme, not just simulated. That is the main reason it was chosen over writing a watchdog by hand for a design that is itself heading to tapeout.

Integration Work Carried Out

#	Step	Detail
1	RTL vendored	`ips/aon_timer/rtl/`: `aon_timer.sv`, `aon_timer_core.sv`, `aon_timer_reg_pkg.sv`, `aon_timer_reg_top.sv`
2	Dependencies vendored	47 support files placed in `SOC_FILES_CENTOS/watchdog/` in their own tree (`prim`, `prim_generic`, `prim_abstract`, `tlul`, `lc_ctrl`, `top_pkg`) so they cannot mix with the existing `register_interface` copies
3	Name-clash resolution	The GPIO already compiles its own `prim_subreg`. The watchdog's copies were renamed to `watchdog_prim_subreg`, `watchdog_prim_subreg_arb` and `watchdog_prim_subreg_ext` (15 and 3 instantiations updated in `aon_timer_reg_top.sv`). No GPIO file was modified
4	Abstract prims generated	Produced with the lowRISC FuseSoC + primgen flow (`fusesoc --cores-root hw/ip hw/top_earlgrey`, target `default`, tool `xcelium`) rather than written by hand, so the wrappers match what the tool emits exactly
5	Standalone verification	Two testbenches written and run before any SoC integration (see below)

Standalone Verification Results

Testbench	Configuration	Result	Coverage
`tb_watchdog.sv`	DIV = 1	64 / 0 **PASS**	Smoke: reset values, register read/write, one bark and one bite
`tb_watchdog_full.sv`	DIV = 1	1 194 / 0 **PASS**	Full register map, wake-up timer, bark, bite, `WDOG_REGWEN` lock, W1C interrupt state, prescaler sweep, pause-in-sleep, `lc_escalate`
`tb_watchdog_full.sv`	DIV = 4	1 194 / 0 **PASS**	as above, slower always-on clock
`tb_watchdog_full.sv`	Asynchronous	1 194 / 0 **PASS**	independent `clk_aon_i`, exercises the register CDC
`tb_watchdog_full.sv`	Seed -662017892	1 184 / 0 **PASS**	randomised prescaler and threshold values

Note

The register CDC back-pressure was measured during these runs at 5 cycles with a matched clock and 11–12 cycles with the divided always-on clock. That measurement is what sizes the APB timeout threshold for the watchdog slot once it is integrated: a 32-cycle threshold gives roughly 3$\times$ margin at the ratios tested, and must be re-derived if the always-on clock is divided further.

Integration Status

Important

The watchdog is verified standalone but is **not yet wired into the SoC**. `NUM_APB_SLAVES` is still 6 and address `0x4000_6000` still decodes as a bus decode error, which the register test confirms. The remaining work is the APB adapter chain (`apb_to_reg` → `reg_to_tlul` → `tlul_cmd_intg_gen` → `aon_timer`), the always-on clock and reset (`rst_aon_ni`), the bark interrupt, the bite reset path, and a reset-cause flag so firmware can tell a watchdog reset from a power-on reset.

Part 2 — Register Access Test

The register test runs firmware on the real core that accesses every register of every peripheral over the complete path:

`core LSU` → `hybrid_mem_interface` → `axi_lite_xbar` → `axi_lite_to_apb` → `IP`

The firmware posts every result to a seven-word buffer at the base of DSRAM, which the testbench watches and files against a golden register map held in the testbench itself. Holding the map on the testbench side rather than in the firmware means a register the firmware forgets to touch is reported as `NOT EXERCISED` instead of quietly disappearing from the totals.

Test Cases

#	Test Case	Method	Registers	Result
1	Reset values	Every register read once, before anything is written	all 57	FAIL
2	Read-only writes ignored	Write ∼reset to each RO register, confirm the value is unchanged and the response is OKAY	8 RO	PASS
3	Read/write pattern sweep	36 patterns per register: 0, all-ones, `0xAAAA_AAAA`, `0x5555_5555` and a walking one, read back through the implemented-bit mask	33 RW	FAIL
4	Per-IP uniqueness	A distinct value written to every R/W register of an IP, then all of them read back; repeated with the inverted values. Catches two offsets decoding to the same storage	33 RW × 2	FAIL
5a	I2C command register	`CR` ignored while `CTR.EN`=0; ACK bit holds; `IACK` self-clears; bus idle afterwards	I2C `CR`, `SR`, `CTR`	PASS
5b	Timer behaviour	Counter advances with `CTRL.EN`=1 and holds with `EN`=0; a `CMP` write clears `VAL`; with `EN`=0 a `VAL`=`CMP` match must *not* clear the counter	both timers, 2 channels	FAIL
5c	UART divisor latch	`DLL`/`DLM` reset values and pattern sweep with `LCR.DLAB`=1; `DLAB` cleared by an `LCR` write; `IER` unaffected	`DLL`, `DLM`, `LCR`, `IER`	FAIL
5d	UART loopback	Two bytes pushed through `THR` → `RBR` with `MCR` loopback; `LSR` data-ready and idle states; `MSR` delta bits clear on read	`THR`, `RBR`, `LSR`, `MSR`, `FCR`, `MCR`	FAIL
5e	GPIO set/clear/toggle	`SET`, `CLEAR` and `TOGGLE` applied to `GPIO_OUT` and confirmed to read back as 0 themselves	4 registers	PASS

#	Test Case	Method	Registers	Result
5f	GPIO input path	GPIO_IN samples the pads through the testbench loopback, and is frozen while GPIO_EN=0	GPIO_IN, GPIO_EN, GPIO_OUT	PASS
5g	GPIO interrupt status	Rising, falling, level-high and level-low events on pin 0; W1C semantics, aggregate status, and that writing 0 clears nothing	9 registers	FAIL
5h	SPI FIFO and reset	Ten words pushed into TXFIFO, level checked, software reset empties both FIFOs	STATUS, TXFIFO	PASS
5i	SPI transfer	64-bit read on chip select 1 against the testbench slave model, both words read out of RXFIFO and compared	CLKDIV, SPICMD, SPIADR, SPILEN, SPIDUM, RXFIFO	PASS
6	Holes and aliases	Nine unmapped offsets inside the decoded windows read 0 and ignore writes; register aliases at +0x040, +0x7C0 and +0xFC0 behave as the same storage	9 holes, 12 aliases	PASS
7	Deliberate faults	15 accesses that must fault: read-only writes that answer PSLVERR, DLAB-protected reads, GPIO sub-word stores, unmapped GPIO offsets and three unmapped MMIO addresses. Each checks the trap count, mcause (5 load / 7 store), mtval, and — from the testbench — whether the response was SLVERR or DECERR, which the core cannot distinguish	15 accesses	PASS
8	Trap accounting	Total traps taken must equal the number of deliberate faults	—	FAIL

Register Coverage

Every register in the map was reached. The address, access kind and number of reported checks are listed below; a check count of n can represent many more bus accesses, since the 36-pattern sweep of test case 3 is reported as a single verdict per register.

Address	Register	Kind	Checks	Result
0x40000000	I2C.PRE	RW	12	FAIL
0x40000004	I2C.CTR	RW	8	FAIL
0x40000008	I2C.RXR	RO	2	PASS
0x4000000C	I2C.SR	RO	3	PASS
0x40000010	I2C.TXR	RW	8	FAIL
0x40000014	I2C.CR	CMD	4	PASS
0x40001000	TIMER0.CH0 VAL	RW	8	FAIL
0x40001004	TIMER0.CH0 CTRL	RW	8	FAIL
0x40001008	TIMER0.CH0 CMP	RW	6	PASS
0x40001010	TIMER0.CH1 VAL	RW	8	FAIL
0x40001014	TIMER0.CH1 CTRL	RW	4	PASS
0x40001018	TIMER0.CH1 CMP	RW	10	FAIL
0x40002000	UART.RBR	RO	3	FAIL
0x40002000	UART.DLL (DLAB=1)	RW	6	FAIL
0x40002004	UART.IER	RW	11	FAIL
0x40002004	UART.DLM (DLAB=1)	RW	2	FAIL
0x40002008	UART.IIR	RO	1	PASS
0x4000200C	UART.LCR	RW	10	FAIL
0x40002010	UART.MCR	RW	8	FAIL
0x40002014	UART.LSR	RO	5	FAIL
0x40002018	UART.MSR	RO	3	FAIL

Address	Register	Kind	Checks	Result
0x4000201C	UART.SCR	RO	2	PASS
0x40003000	GPIO.INFO	RO	2	PASS
0x40003004	GPIO.CFG	RW	8	FAIL
0x40003008	GPIO.GPIO_MODE_0	RW	8	FAIL
0x4000300C	GPIO.GPIO_MODE_1	RW	8	FAIL
0x40003080	GPIO.GPIO_EN	RW	8	FAIL
0x40003100	GPIO.GPIO_IN	RO	5	PASS
0x40003180	GPIO.GPIO_OUT	RW	9	FAIL
0x40003200	GPIO.GPIO_SET	CMD	3	PASS
0x40003280	GPIO.GPIO_CLEAR	CMD	3	PASS
0x40003300	GPIO.GPIO_TOGGLE	CMD	3	PASS
0x40003380	GPIO.INTRPT_RISE_EN	RW	8	FAIL
0x40003400	GPIO.INTRPT_FALL_EN	RW	8	FAIL
0x40003480	GPIO.INTRPT_LVL_HIGH_EN	RW	8	FAIL
0x40003500	GPIO.INTRPT_LVL_LOW_EN	RW	8	FAIL
0x40003580	GPIO.INTRPT_STATUS	W1C	5	FAIL
0x40003600	GPIO.INTRPT_RISE_STATUS	W1C	4	PASS
0x40003680	GPIO.INTRPT_FALL_STATUS	W1C	3	PASS
0x40003700	GPIO.INTRPT_LVL_HIGH_STATUS	W1C	3	PASS
0x40003780	GPIO.INTRPT_LVL_LOW_STATUS	W1C	3	PASS
0x40004000	SPI.STATUS	CMD	6	PASS
0x40004004	SPI.CLKDIV	RW	10	FAIL
0x40004008	SPI.SPICMD	RW	8	FAIL
0x4000400C	SPI.SPIADR	RW	8	FAIL
0x40004010	SPI.SPILEN	RW	8	FAIL
0x40004014	SPI.SPIDUM	RW	8	FAIL
0x40004018	SPI.TXFIFO	WO	2	PASS
0x40004020	SPI.RXFIFO	RO	2	PASS
0x40004024	SPI.INTCFG	RW	8	FAIL
0x40004028	SPI.INTSTA	RO	1	PASS
0x40005000	TIMER1.CH0 VAL	RW	8	FAIL
0x40005004	TIMER1.CH0 CTRL	RW	8	FAIL
0x40005008	TIMER1.CH0 CMP	RW	10	FAIL
0x40005010	TIMER1.CH1 VAL	RW	8	FAIL
0x40005014	TIMER1.CH1 CTRL	RW	8	FAIL
0x40005018	TIMER1.CH1 CMP	RW	10	FAIL

Metric	Value	Note
Registers in the golden map	57	I2C 6, Timers 12, UART 10, GPIO 19, SPI 10
Registers exercised	57	none missed
Registers passing	20	
Registers failing	37	single root cause, see below
Firmware checks	260 passed, 180 failed	
Testbench checks	26 passed, 3 failed	
Traps taken	2 311	15 expected
Runtime	325 553 cycles after boot	12 985 MMIO reads, 652 writes

Root Cause of the Failures

The failures are not register defects and not caused by the bus monitors — no monitor fired at any point during the run. They all trace to a single defect in the core, first exposed by this test.

Important

Address corruption in the core. Every failing case is in test 3 or 4, where the address is computed and passed to a helper function. The call site compiles to:

```
108cc:  lw  a0,4(s2)     ; a0 = ip->base
108d0:  lw  a2,4(a1)     ; a2 = r->off
108e0:  add a2,a0,a2     ; a2 = base + off
108f0:  jalr ra,-2040(ra) ; call
```

For `I2C.PRE` the result must be `0x4000_0000`. The register-file dump taken at the faulting access shows `a2 = 0x4002_02b4`, that is `0x4000_0000 + 0x0002_02b4`, where `0x0002_02b4` is a DSRAM pointer inside the register tables and not a register offset. The whole 36-pattern loop then hammers that one bogus address, every access decodes as a bus error, the core traps, the load is skipped and the read-back compares against nothing — which is why the failures all read as `got 0x00000000`.
Two instructions earlier there is a load-use hazard whose consumer is a conditional branch (`lw a5,8(a1)` followed immediately by `bne a5,s5`). That is the first place to look in `hazard_unit.v` and the Decode/Execute forwarding path.

Everything that does not go through that code path passes: reset values, read-only rejection, address holes, aliases, all 15 deliberate faults with correct `mcause` and `mtval`, the GPIO output path, and the full SPI transfer. Two further failures are independent of the core defect and are stale-RTL issues on the simulation machine — the timer IRQ is not yet qualified with `CTRL.EN` there, and the GPIO W1C race fix in `prim_subreg_arb.sv` has not been copied across.

Part 3 — Bus Timeout Monitors

An APB slave that never asserts `PREADY` stops the bridge, which stops the crossbar, which stops the CPU, with no way out except a power-on reset. The concrete case is reading the SPI master's RX FIFO while it is empty. Two monitors were added to bound this:

Module	Function
`apb_timeout_monitor`	One per APB slot. Counts the cycles a slave spends in the ACCESS phase without asserting `PREADY`; past its threshold it answers the *bridge* with `PREADY=1`, `PSLVERR=1` and `0xDEADBEEF`. The slave's own `PREADY` is never driven, so an aborted SPI FIFO read pops nothing
`axi_lite_timeout_monitor`	One instance on the core's AXI-Lite master port. Backstop for anything the APB monitors cannot see. On expiry it answers the core with SLVERR, keeps driving the stuck request downstream until the crossbar accepts it, swallows the late response, and error-answers new requests while draining

`axi_lite_to_apb` turns `PSLVERR` into an AXI SLVERR, which `hybrid_mem_interface` turns into a precise load or store access fault — `mcause` 5 or 7 with `mtval` set to the faulting address. Firmware therefore sees a normal exception instead of a dead machine.

Peripheral Timing, Thresholds and Hang Mechanisms

Each threshold is sized from the longest *legitimate* wait that slot can present in a single APB ACCESS phase — not from the length of a peripheral transfer. A transfer of n words is n separate APB accesses and never approaches the threshold.

Slot / IP	TX/RX time (system clocks)	Time-out	Margin	What can hang this slot
0 I2C	SCL period $= 5(\mathrm{PRER}+1)$; one byte + ACK $= 9$ periods $= 9\,000$ at 100 kHz. Register access: 0 wait states	16	$16\times$	`PREADY` is tied high in `apb_i2c.sv`, so the slave cannot stall the bus at all. The monitor is a fault detector here — it catches a stuck decoder or a clock/reset fault. The real risk is firmware polling `SR.TIP` forever after an arbitration loss, which is a software hang, not a bus hang, and is what the watchdog covers
1 Timer 0	No serial stream. Register access: 0 wait states	16	$16\times$	`PREADY` tied high in `timer.sv`. Fault detector only, as above
2 UART	Bit $= 16\times$ divisor; one 8N1 character $= 160\times$ divisor $= 8\,640$ at 115 200 baud. Register access: 1 wait state through `apb_to_obi`	16	$8\times$	`PREADY` is produced by an OBI round trip. If the OBI response never returns — the peripheral held in reset, where `gnt` is combinational but `rvalid` is a flop stuck at 0 — `PREADY` stays low forever and the access never completes. This is a real hang path, not a hypothetical one
3 GPIO	No serial stream. Register access: feedthrough, 0 wait states	16	$16\times$	`apb_to_reg` feedthrough. A `reg_bus` that never asserts ready hangs the access. Sub-word writes are already blocked by the wrapper and answered with `PSLVERR` rather than stalling
4 SPI Master	Bit $= 2(\mathrm{CLKDIV}+1)$; one 32-bit word $= 64(\mathrm{CLKDIV}+1)$. First word of a burst adds command, address and dummy bits. 640 at CLKDIV=4; 32 768 at CLKDIV=255; worst legitimate first-word wait 36 864	65 536	$1.78\times$	**The one slot that stalls in normal operation.** Reading `RXFIFO` while it is empty, or writing `TXFIFO` while it is full, with no transfer running, holds `PREADY` low indefinitely. A wrong configuration or a mistimed access is enough. This is the defect the whole exercise exists for
5 Timer 1	As Timer 0	16	$16\times$	As Timer 0
AXI SRAM region	Fixed latency, roughly 4 cycles	64	$16\times$	Crossbar or SRAM wrapper deadlock. Not reachable from the core today: `hybrid_mem_interface` sends only `0x4000_0000`–`0x4FFF_FFFF` over AXI and handles DSRAM directly, so every core transaction decodes as the APB region
AXI APB region	Bounded by the APB slot timeout above	131 072	$2\times$	A bridge or crossbar deadlock upstream of `PSEL`. If `axi_lite_to_apb` stalls in its Setup state — for example with its response registers backed up — it never drives `PSEL`, so no APB monitor can see the transaction and only the AXI backstop can end it

Note

The SPI threshold is deliberately sized per *word*, not per transfer. At the worst divider a single word takes 32 768 clocks and the first word of a burst 36 864, so 65 536 keeps 1.78× margin; an 8 192-word flash read is 8 192 separate APB accesses and never gets close. The AXI APB-region threshold is exactly $2\times$ the largest APB slot threshold, so the per-slot monitor always answers first and the backstop only fires for faults the slot monitors cannot see.

Test Cases

Only the SPI master can stall by itself. For the other slots the firmware asks the testbench, through its result buffer in DSRAM, to hold that slave's `PREADY` low; for the AXI cases it asks for the bridge to be frozen before it drives `PSEL`. DSRAM is handled directly by `hybrid_mem_interface`, so this channel still works while the bus is jammed.

#	Test Case	What Is Checked	Cases	Result
1	Timeout, per slot	The counter runs to the threshold and the access is answered with SLVERR; `mcause` 5, `mtval` equal to the address; elapsed cycles inside the expected window	6	PASS

#	Test Case	What Is Checked	Cases	Result
2	Blocked path, per slot	While the slave is still stuck the next access fails immediately instead of paying the threshold again, measurably faster than the first abort	6	PASS
3	Store abort, per slot	The same abort on a write, with `mcause` 7	6	PASS
4	Recovery, per slot	After the stall is released the slot reads and writes normally at its usual latency	6	PASS
5	SPI empty RX FIFO	A read of an empty `RXFIFO` aborts after 65 536 cycles and the FIFO levels are unchanged, proving no data was popped	1	PASS
6	SPI block clears	A second empty read pays the full threshold again, because the SPI master releases `PREADY` when deselected — the documented difference from a slave stuck low permanently	1	PASS
7	SPI full TX FIFO	An eleventh word written to a full `TXFIFO` aborts with `mcause` 7 and is not accepted	1	PASS
8	SPI legitimate wait	At CLKDIV=255 with command and address phases, a blocking `RXFIFO` read issued before the first word exists waits 32 768 cycles, must *not* abort, and returns the correct data	1	PASS
9	AXI read timeout	With the bridge frozen, a read aborts at 131 072 cycles and *no* APB monitor fires, proving the backstop covers what the slot monitors cannot see	1	PASS
10	AXI write timeout	The same for a write, `mcause` 7	1	PASS
11	Drain behaviour	While the orphaned transaction is still outstanding, a new access is error-answered immediately rather than forwarded into the collision	2	PASS
12	Orphan semantics	After release, the write that the fabric had already accepted does reach the slave, while the one answered during drain does not	1	PASS
13	No false timeout, per slot	150 back-to-back accesses per slot with no idle cycle; no monitor may fire	6	PASS
14	No false timeout, loaded	The same with Timer 0 counting, the UART transmitting and an SPI transfer in flight; peripheral data intact afterwards	1	PASS

Results

Monitor	Threshold	Aborts	Expected	Checks	Result
APB[0] I2C	16	3	3	19	PASS
APB[1] Timer 0	16	3	3	19	PASS
APB[2] UART	16	3	3	19	PASS
APB[3] GPIO	16	3	3	19	PASS
APB[4] SPI	65 536	6	6	44	PASS
APB[5] Timer 1	16	3	3	19	PASS
AXI-Lite core port	131 072	2	2	23	PASS
Cross-peripheral traffic	—	0	0	6	PASS

Metric	Value	Note
Firmware checks	168 passed, 0 failed	matches the expected total
Testbench checks	153 passed, 0 failed	matches the expected total

Metric	Value	Note
Access faults taken	25	25 expected, all with correct `mcause/mtval`
Monitor aborts	23	21 APB + 2 AXI
Sticky status at end	`0xFF`	every monitor exercised
Measured AXI abort latency	131 070 cycles	threshold − 2, as expected
Runtime	598 234 cycles after boot	1 029 MMIO reads, 367 writes

Beyond the pass/fail totals, the testbench verifies independently of the RTL that each abort happened at exactly the configured threshold, that no monitor ever fired in a cycle where the slave was asserting `PREADY`, that on every abort the bridge saw `PREADY+PSLVERR` while the slave side was still low, and that on both sides of the AXI monitor every read address was answered by exactly one read response and every write address by exactly one write response — so no orphaned transaction leaked through and no injected response was duplicated.

Note

Finding. Aborting a UART access trips the UART's own protocol assertion: `apb_to_obi.penable: APB PENABLE must be asserted during OBI RESP phase`. That assertion assumes an APB master never abandons a transfer, and abandoning transfers is exactly what the monitor does. It is not a testbench artefact — it is what a UART timeout will look like in silicon, and it means the OBI bridge is left mid-handshake and will need a reset to recover.

Outstanding Items

#	Item	Priority	Description
1	Core address corruption	High	Root-cause and fix the defect exposed by register test case 3; suspect the load-use hazard feeding a conditional branch in `hazard_unit.v`. Until it is fixed the register test cannot sign off
2	Watchdog SoC integration	High	APB adapter chain, slot 6 at `0x4000_6000`, always-on clock and reset, bark interrupt, bite reset and a reset-cause flag
3	Boot-time SPI hang	Medium	The bootloader's `S_READ_FIFO` state waits on `PREADY` through its own master port, bypassing the monitor. A stall there is still unrecoverable without a power-on reset
4	UART abort recovery	Medium	Decide whether `apb_to_obi` should be made abort-tolerant or whether a UART timeout is allowed to require a peripheral reset
5	Stale RTL on the simulation host	Medium	Copy the qualified-IRQ fix in `timer.sv` and the W1C race fix in `prim_subreg_arb.sv`
6	Monitor unit testbenches	Low	Standalone benches for `apb_timeout_monitor` and `axi_lite_timeout_monitor`, independent of the SoC
7	Remaining filelists	Low	`soc_timer0_boot_run.f`, `soc_imem_boot.f` and `soc_imem_boot_vendor.f` still need the two monitor source lines